AdaBoost Ensemble for Bank Marketing Campaign ROI

Cameron Batts — AdaBoostClassifier · One-Hot Encoding · Custom Value Function

In [5]:
```
import numpy as np
import pandas as pd
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import make_scorer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

In [6]:
```
bank_data = pd.read_csv("bank_data.csv")
bank_data.head()
```

Out[6]:

	age;"job";"marital";"education";"default";"balance";"housing";"loan";"contact
0	
1	
2	
3	
4	

In [7]:
```
bank_data = pd.read_csv("bank_data.csv", sep=";")
bank_data.head()
```

Out[7]:

	age	job	marital	education	default	balance	housing	loan	cont
0	58	management	married	tertiary	no	2143	yes	no	unkn
1	44	technician	single	secondary	no	29	yes	no	unkn
2	33	entrepreneur	married	secondary	no	2	yes	yes	unkn
3	47	blue-collar	married	unknown	no	1506	yes	no	unkn
4	33	unknown	single	unknown	no	1	no	no	unkn

In [8]:
```
try:
```

```
    bank_data.drop(['day', 'month', 'duration', 'pdays', 'poutcome'],
except NameError:
    print('The object `bank_data` does not exist!')
```

Here, we encode the response variable (whether or not a customer subscribed a term deposit with the bank following a marketing campaign) with a binary indicator.

In [9]:
```
try:
    bank_data["y"] = bank_data['y'].apply(lambda y: 1 if y == 'yes' el
    bank_data.head()
except NameError:
    print('The object `bank_data` does not exist!')
```

In [10]:
```
bank_data = pd.get_dummies(bank_data)
bank_data.head()
```

Out[10]:

	age	balance	campaign	previous	y	job_admin.	job_blue-collar	job_entreprene
0	58	2143	1	0	0	False	False	Fal
1	44	29	1	0	0	False	False	Fal
2	33	2	1	0	0	False	False	Tr
3	47	1506	1	0	0	False	True	Fal
4	33	1	1	0	0	False	False	Fal

5 rows × 33 columns

Let's now separate the predictors from the response variable.

In [11]:
```
try:
    X = bank_data.drop('y', axis=1)
    Y = bank_data['y']
except NameError:
    print('The object `bank_data` does not exist!')
```

In [12]:
```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, random_state
```

We want to use the available data to build a predictive model that can assist us in making our next marketing campaign more efficient.

Here are some quantities to keep in mind. Our department estimates that:

- a marketing contact with a potential customer costs around 10 Euros on average

- a successful contact (i.e. the customer subscribes a term deposit) generates on average 100 Euros of profits for the bank (say, present value net the cost of marketing).

Accordingly, we estimate that:

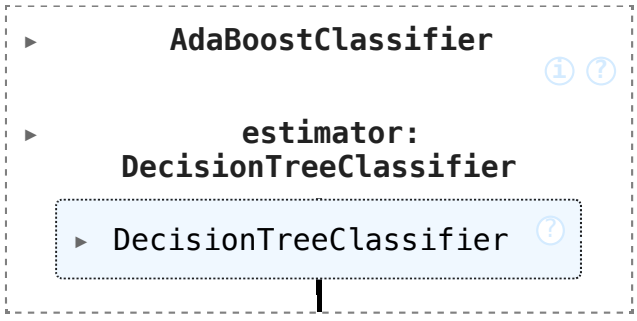
- the value associated with a true negative prediction from our model is 10 Euros (it saves us the waste of 10 Euros associated with the marketing contact)
- the value associated with a false positive prediction from our model is -10 Euros
- the value associated with a false negative prediction from our model is -100 Euros
- the value associated with a true positive prediction from our model is +100 Euros.

Let's encode this information in a "value function" that we will use later.

In [13]:
```
def value_function(y_true, y_pred, tn_value=10, fp_value=-10, fn_value
    sum_ = y_pred + y_true
    diff_ = y_pred - y_true
    tn_contrib = tn_value * np.mean((sum_ == 0) & (diff_ == 0))
    fp_contrib = fp_value * np.mean((sum_ == 1) & (diff_ == 1))
    fn_contrib = fn_value * np.mean((sum_ == 1) & (diff_ == -1))
    tp_contrib = tp_value * np.mean((sum_ == 2) & (diff_ == 0))
    return tn_contrib + fp_contrib + fn_contrib + tp_contrib
```

In [14]:
```
ada_boost = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(random_state=42, max_depth=5),
    n_estimators=2000,
    learning_rate=0.80,
    random_state=42)
ada_boost.fit(X_train, Y_train)
```

Out[14]:



We will also fit a conventional decision tree for reference.

In [15]:
```
try:
    tree = DecisionTreeClassifier(random_state=42).fit(X_train, Y_trai
except NameError:
    print('The objects `X_train, Y_train` do not exist!')
```

In [16]:
```
ada_value = value_function(Y_test.values, ada_boost.predict(X_test))
tree_value = value_function(Y_test.values, tree.predict(X_test))

print('AdaBoost value function:', ada_value)
print('Decision Tree value function:', tree_value)

# the decision tree scored higher so it looks like its doing better he
```

```
AdaBoost value function: -0.7431655312748837
Decision Tree value function: 0.9926568167743075
```

Let's now try to quantify what is the monetary impact of using the `AdaBoostClassifier` as opposed to the `DecisionTreeClassifier` on our marketing campaign.

First off, for the evaluation of a given model, we will assume that in our marketing campaign we will only contact customers that are predicted as subscribers by our model.

With this in mind, let's create a "marketing campaign profit function".

In [17]:
```
def marketing_profits(model, X, Y, fp_value=-10, tp_value=100):
    tp_contrib = np.sum((model.predict(X) > 0) & (Y > 0)) * tp_value
    fp_contrib = np.sum((model.predict(X) > 0) & (Y < 1)) * fp_value
    return tp_contrib + fp_contrib
```

In [18]:
```
ada_prof = marketing_profits(ada_boost, X_test, Y_test)
tree_prof = marketing_profits(tree, X_test, Y_test)
percent_diff = ((ada_prof - tree_prof) / tree_prof) * 100
print('Percent change in profits using AdaBoost vs Decision Tree:', pe
```

```
Percent change in profits using AdaBoost vs Decision Tree: -41.72692471
288813 %
```

In [19]:
```
def _value_function(y, y_pred, **kwargs):
    return value_function(y, y_pred, **kwargs)


value_function_wrapper = make_scorer(_value_function)
```

In [20]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

# for my model i went with random forest since its basically just a lo
# working together which should hopefully do better than one tree alon
```

```
# the data is imbalanced (only like 10% said yes) so im using class_we
# to make the model not just ignore the yes group

rf = RandomForestClassifier(class_weight='balanced', random_state=42)

# trying out a few different values for number of trees and max depth
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, None]
}

# need to set up the value function as a scorer so GridSearchCV can us
def _value_function(y, y_pred, **kwargs):
    return value_function(y, y_pred, **kwargs)

value_function_wrapper = make_scorer(_value_function)

# running grid search with 5 fold cv using our value function as the m
# this way the model is tuned based on what actually matters for this
grid_search = GridSearchCV(rf, param_grid, scoring=value_function_wrap
grid_search.fit(X_train, Y_train)

print('Best parameters:', grid_search.best_params_)
print('Best CV score:', grid_search.best_score_)

best_rf = grid_search.best_estimator_

# now comparing all 3 models on the test set with the value function
rf_val = value_function(Y_test.values, best_rf.predict(X_test))
ada_val = value_function(Y_test.values, ada_boost.predict(X_test))
tree_val = value_function(Y_test.values, tree.predict(X_test))

print('\nValue Function Results:')
print('AdaBoost:', ada_val)
print('Decision Tree:', tree_val)
print('Random Forest (tuned):', rf_val)

# and now checking the marketing profits for each model
rf_prof = marketing_profits(best_rf, X_test, Y_test)
ada_prof = marketing_profits(ada_boost, X_test, Y_test)
tree_prof = marketing_profits(tree, X_test, Y_test)

print('\nMarketing Profits:')
print('AdaBoost:', ada_prof)
print('Decision Tree:', tree_prof)
print('Random Forest (tuned):', rf_prof)

# percent change vs decision tree
rf_vs_tree = ((rf_prof - tree_prof) / tree_prof) * 100
print('\nRF vs Decision Tree profit change:', round(rf_vs_tree, 2), '%
```

```
# percent change vs adaboost
rf_vs_ada = ((rf_prof - ada_prof) / ada_prof) * 100
print('RF vs AdaBoost profit change:', round(rf_vs_ada, 2), '%')
```

```
Best parameters: {'max_depth': 5, 'n_estimators': 200}
Best CV score: 7.150506182578337

Value Function Results:
AdaBoost: -0.7431655312748837
Decision Tree: 0.9926568167743075
Random Forest (tuned): 6.952136600902416

Marketing Profits:
AdaBoost: 13700
Decision Tree: 23510
Random Forest (tuned): 57190

RF vs Decision Tree profit change: 143.26 %
RF vs AdaBoost profit change: 317.45 %
```